PANDAS AND DATA DISCOVERY



- DATA DATA EVERYWHERE…..IN WALKS PANDAS
- GEOSPATIAL DATA
- PRACTICAL 6



David Topping: david.topping@manchester.ac.uk

'Vanilla' Python

Data types
Basic operations

Loops
Conditionals

Pandas and data discovery

Geospatial analysis

Statistical Analysis

Cluster Analysis

1 2 3 4 5 6 7 8 9

The Numpy module

Conway's Game Of Life

Differential Equations

Modules and Functions

Creating a notebook assessment
select topic from weeks 4 - 9

10 11 12

COURSE LAYOUT >>>>

LECTURE + PRACTICAL. SOLUTIONS ARE GIVEN

RECAP

- Python comes in a few flavours. As we move away from Vanilla Python, we start using modules that offer a number of benefits: Quicker operations, better sharing...

- We can store information in arrays. These can be 1D, 2D, 3D, 4D. Indexing starts at 0.

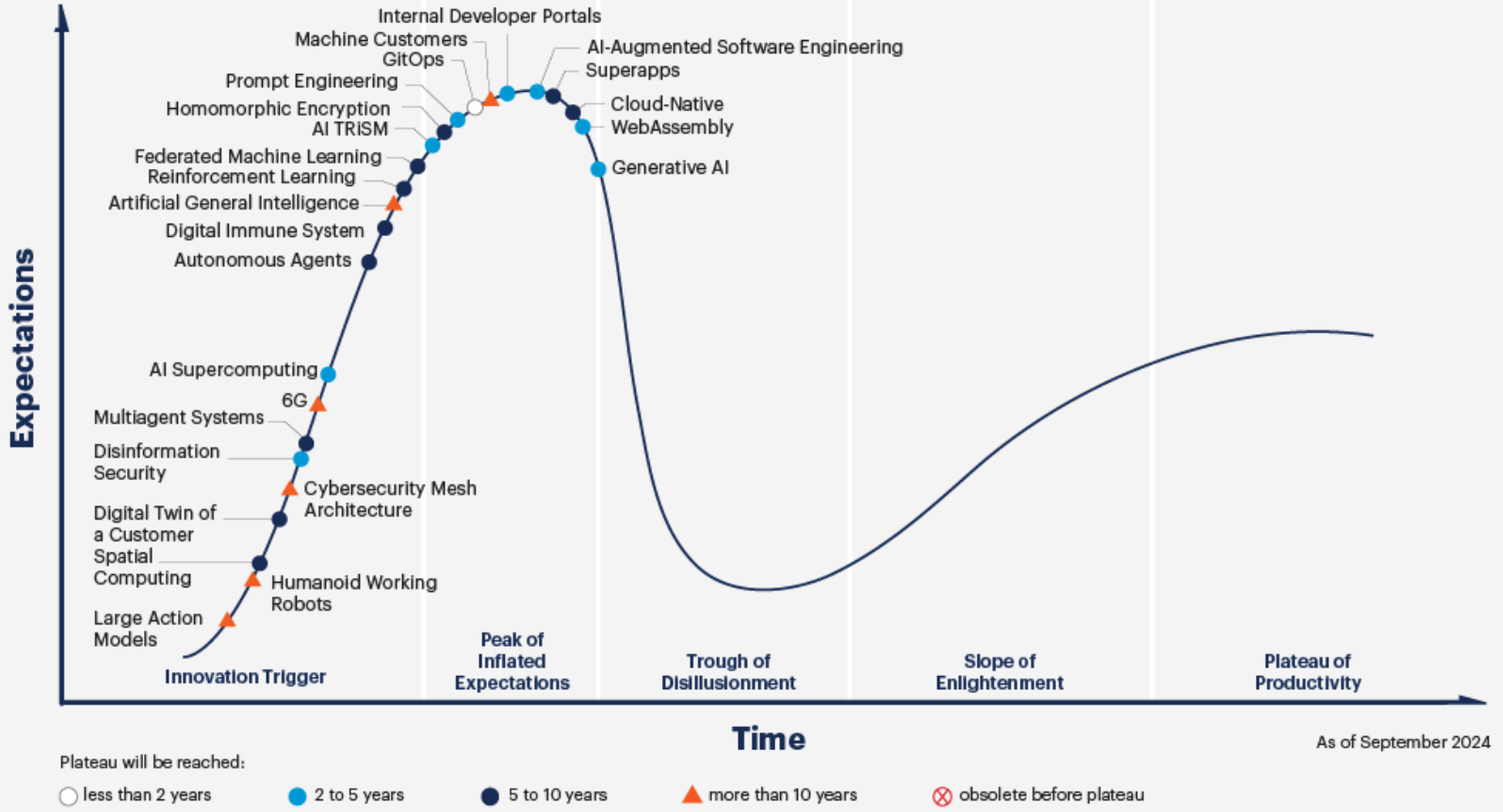
- Functions help us repeat a set of operations many times

- We can create and use numerical models.
  - These can be built using existing 'solvers'
  - Traditional numerical models are built on differential equations

What if we have data we want to load in?

DATA DATA EVERYWHERE...

Hype Cycle for Emerging Technologies, 2024

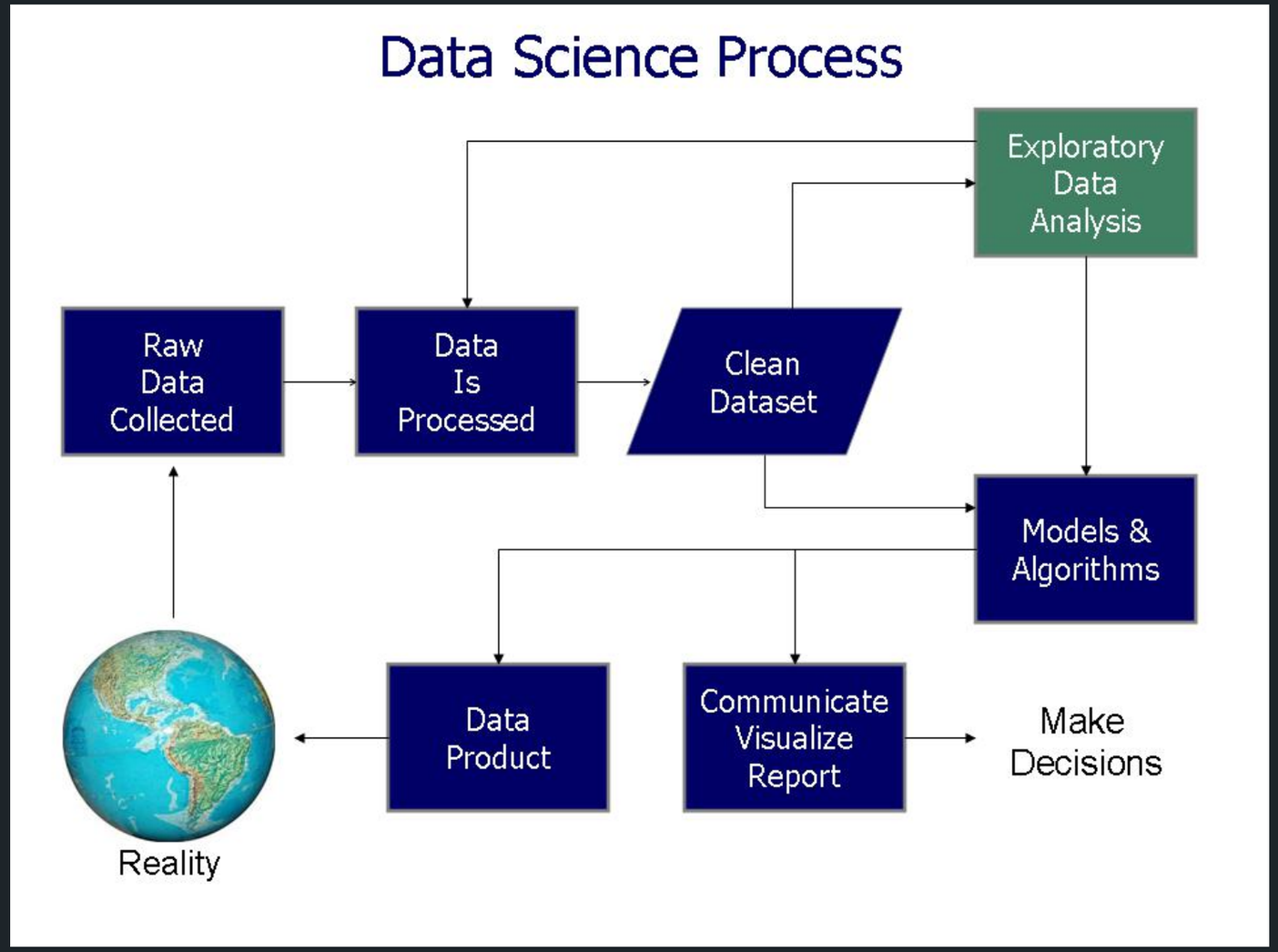


Source: Gartner

Commercial reuse requires approval from Gartner and must comply with the Gartner Content Compliance Policy on gartner.com.
© 2024 Gartner, Inc. and/or its affiliates. All rights reserved. 3205434

Gartner

DATA DATA EVERYWHERE...

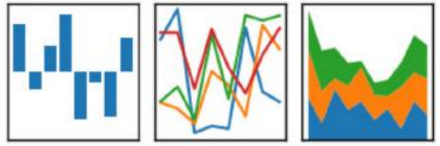


Doing Data Science, O'Reilly ISBN: 978-1-449-35865-5. ↑ Forbes-Gil Press-A Very Short History of Data Science-May 2013

IN WALKS PANDAS

pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



"pandas...provides high-performance, easy-to-use data structures and data analysis tools.."

Format Type	Data Description	Reader	Writer
text	CSV	read_csv	to_csv
text	JSON	read_json	to_json
text	HTML	read_html	to_html
text	Local clipboard	read_clipboard	to_clipboard
binary	MS Excel	read_excel	to_excel
binary	HDF5 Format	read_hdf	to_hdf
binary	Feather Format	read_feather	to_feather
binary	Parquet Format	read_parquet	to_parquet
binary	Msgpack	read_msgpack	to_msgpack
binary	Stata	read_stata	to_stata
binary	SAS	read_sas	
binary	Python Pickle Format	read_pickle	to_pickle
SQL	SQL	read_sql	to_sql
SQL	Google Big Query	read_gbq	to_gbq

FILE FORMATS CAN VARY E.G. JSON FILE

{} sample.json > [ ] data

```json
{
    "data": [
        {
            "type": "articles",
            "id": "1",
            "attributes": {
                "title": "Working with JSON Data in python",
                "description": "This article explains the various ways to work with JSON data in python.",
                "created": "2020-12-28T14:56:29.000Z",
                "updated": "2020-12-28T14:56:28.000Z"
            },
            "author": {
                "id": "1",
                "name": "Aveek Das"
            }
        }
    ]
}
```

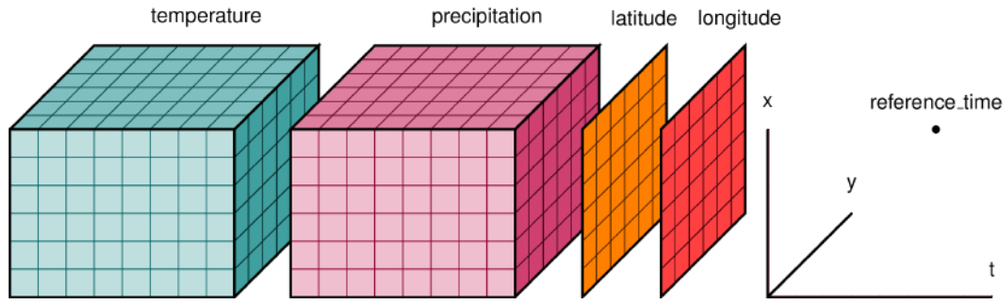
FILE FORMATS CAN VARY E.G. NETCDF
A NETCDF FILE HAS DIMENSIONS, VARIABLES, AND ATTRIBUTES.

```
netcdf ./data/NARR_tasmax_201701.nc {
  dimensions:
x = 349;
y = 277;
time = 31;
  variables:
float lon(y=277, x=349);
  :units = "degrees_east";
  :long_name = "Longitude";
  :CoordinateAxisType = "Lon";

float lat(y=277, x=349);
  :units = "degrees_north";
  :long_name = "Latitude";
  :CoordinateAxisType = "Lat";

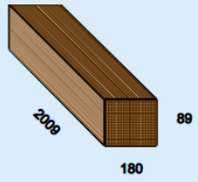
double time(time=31);
  :long_name = "Time";
  :delta_t = "";

float tasmax(time=31, y=277, x=349);
  :long_name = "Daily maximum temperature";
  :units = "Celcius";
  :missing_value = -999.0; // double
  :coordinates = "lon lat";
```



xarray.DataArray 'sst' (**time**: 2009, **lat**: 89, **lon**: 180)

	Array	Chunk
Bytes	122.77 MiB	196.19 kiB
Shape	(2009, 89, 180)	(2009, 5, 5)
Count	648 Tasks	648 Chunks
Type	float32	numpy.ndarray



▾ Coordinates:

lat	(lat)	float32	88.0 86.0 84.0 ... -86.0 -88.0
lon	(lon)	float32	0.0 2.0 4.0 ... 354.0 356.0 358.0
time	(time)	datetime64[ns]	1854-01-01 ... 2021-05-01

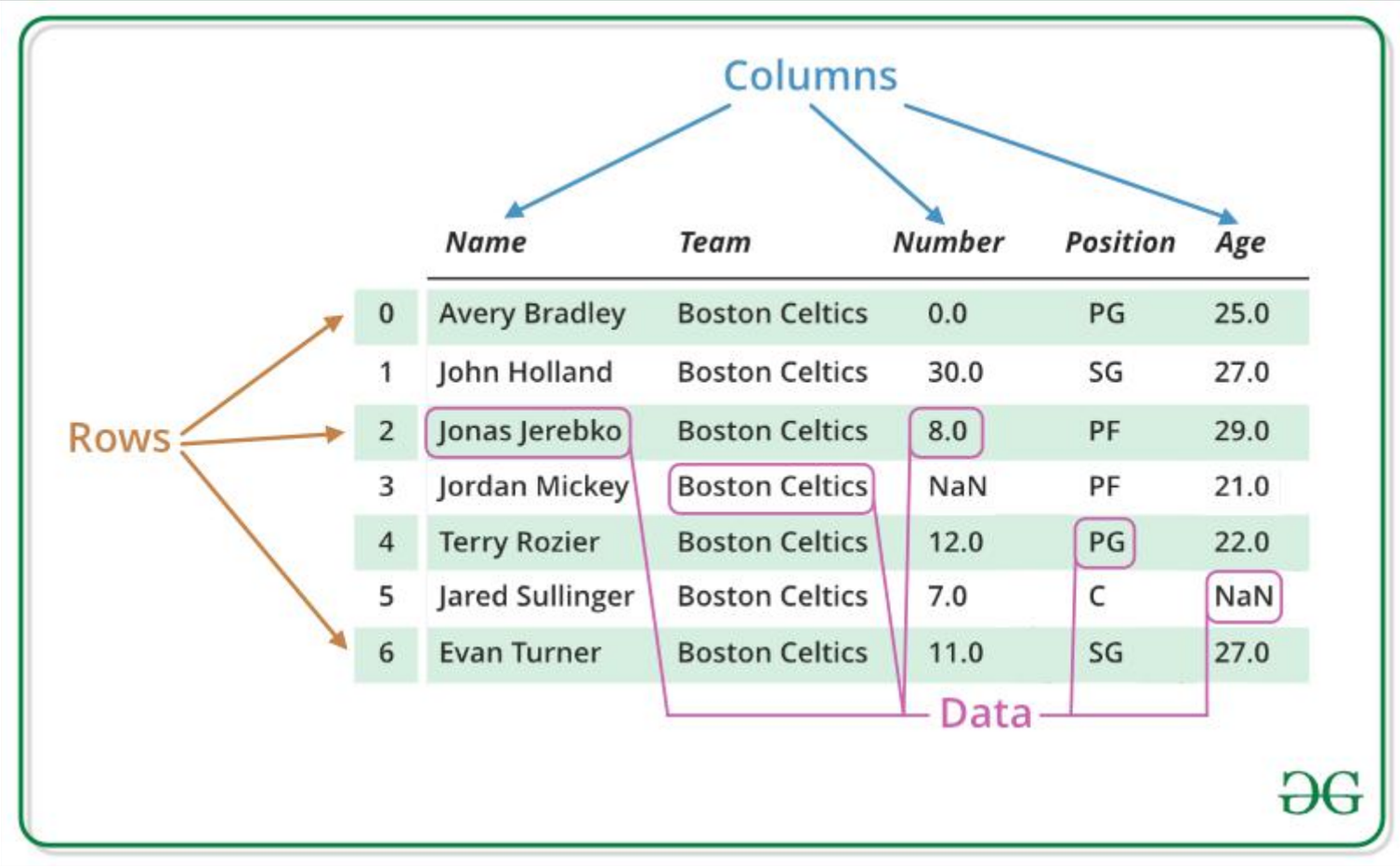
▾ Attributes:

long_name :	Monthly Means of Sea Surface Temperature
units :	degC
var_desc :	Sea Surface Temperature
level_desc :	Surface
statistic :	Mean
dataset :	NOAA Extended Reconstructed SST V5
parent_stat :	Individual Values
actual_range :	[-1.8 42.32636]
valid_range :	[-1.8 45.]

https://blog.csdn.net/kbasg

IN WALKS PANDAS

```
import pandas as pd
```



"A Pandas DataFrame is two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns).
A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. "

IN WALKS PANDAS

```
#Lets load in some data using the Pandas package. First we need to import that package.
import pandas as pd #Im using pd here as its easier to keep writing! You can use whatever you want, but it might

# Read data from file 'GVP_Eruption_Results.csv'
# We are going to use the function 'read_csv' within the Pandas package:


# Notice how we call that function using the '.' operator?
# (Note the data file needs to be in the same directory that your jupyter notebook is based) You can control deli
data = pd.read_csv("data/GVP_Eruption_Results.csv")

# How do we preview the data file.
# Preview the first 5 lines of the loaded data
data.head()

#Pandas has loaded the data into a dataframe. By using the extension .head() we can take a look at the data produ
#available. At the moment this looks like a matrix where the rows are simply data entries, rather than ordered by
#a particular variable, such as time of eruption or eruption number. At least we can scan the range of column hea
#and think about what we want to do next. By the way this is also a good way of ensuring you have loaded in the c
#you expect. Sound silly? If you work with lots of datafiles it can be easy to assume you are working on the corr
#one
```

	Volcano Number	Volcano Name	Eruption Number	Eruption Category	Area of Activity	VEI	VEI Modifier	Start Year Modifier	Start Year	Start Year Uncertainty	...	Evidence Method (dating)	End Year Modifier	End Year	End Year Uncertainty	End Month	En Da Modifie
0	252120	Ulawun	22319	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	NaN	2019.0	NaN	6.0	Na
1	354020	Ubinas	22324	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	>	2019.0	NaN	7.0	Na
2	290250	Raikoke	22321	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	NaN	2019.0	NaN	7.0	Na
3	233020	Fournaise, Piton de la	22318	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	NaN	2019.0	NaN	6.0	Na
4	311120	Great Sitkin	22322	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	NaN	2019.0	NaN	6.0	Na

ACCESS PANDAS FUNCTIONS USING THE '.' OPERATOR

ACCESS DATAFRAME FUNCTIONS/PROPERTIES USING THE '.' OPERATOR

INTERROGATING PROPERTIES OF THE DATAFRAME

E.G SIZE AND SHAPE

```
#dataframe.size - as in rows x columns
size = data.size
print("Size of dataframe =", size)

# dataframe.shape - gives the seperated number of rows, columns, etc
shape = data.shape
print("Shape of dataframe =", shape)

# dataframe.ndim  - Number of dimensions. Do we have a 2D array, 3D array?? etc
ndim = data.ndim
print("Number of dimensions =", ndim)
```

```
Size of dataframe = 237384
Shape of dataframe = (9891, 24)
Number of dimensions = 2
```

INTERROGATING PROPERTIES OF A COLUMN

E.G SIZE AND SHAPE

```
# series.ndim  - Again im using my own variable name here as 'series'.
series_ndim = data["Eruption Category"].ndim
print("Number of dimensions =", series_ndim)

series_size = data["Eruption Category"].size
print("Shape of dataframe =", series_size)

series_shape = data["Eruption Category"].shape
print("Size of dataframe =", series_shape)
```

```
Number of dimensions = 1
Shape of dataframe = 9891
Size of dataframe = (9891,)
```

INTERROGATING PROPERTIES OF A COLUMN

E.G. UNIQUE ENTRIES

```
data['Volcano Name'].value_counts()
```

```
Etna                              196
Fournaise, Piton de la            187
Asosan                            171
Villarrica                        152
Katla                             128
                                 ...
Longgang Group                      1
East Gakkel Ridge at 85°E           1
Chiles-Cerro Negro                  1
Imuruk Lake                         1
Aliso                               1
Name: Volcano Name, Length: 862, dtype: int64
```

EXTRACT VALUES FROM A COLUMN

ENGLISH ⟶ PYTHON

We have a dataframe called 'data'

We have a column labelled 'Start Year'

We want to extract values from this column into a numpy array

We want to find the minimum value in this array

```
Years = data['Start Year'].values
Earliest_eruption = np.min(Years)
```

EXTRACT VALUES FROM A COLUMN

ENGLISH ⟶ PYTHON

We have a dataframe called 'data'

We have columns labelled 'Latitude' and 'Longitude'

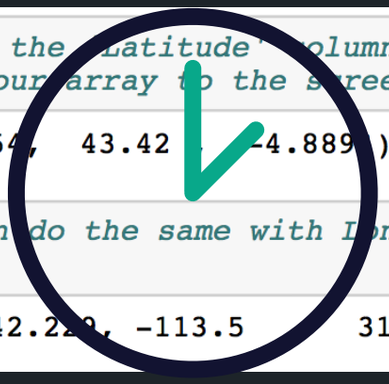
We want to extract values from this column into a numpy array

```
Latitude = data['Latitude'].values # Extract the 'Latitude' column and th
Latitude #This will just print a preview of our array to the screen.
```

```
array([ -5.05 , -16.355,  48.292, ...,  38.654,  43.42 , -4.889])
```

```
Longitude = data['Longitude'].values # We can do the same with Longitude
Longitude
```

```
array([ 151.33 ,  -70.903,  153.25 , ...,   42.22 , -113.5  ,  31.933])
```



EXTRACT A SUBSET - USING CONDITIONALS!

e.g preview only those entries where 'Start Year' == 2019

Notice the use of square brackets [ ] and the conditional within them

```
data[data['Start Year'] == 2019].head()
```

	Volcano Number	Volcano Name	Eruption Number	Eruption Category	Area of Activity	VEI	VEI Modifier	Start Year Modifier	Start Year	Start Year Uncertainty	...	Evidence Method (dating)	End Year Modifier	End Year
0	252120	Ulawun	22319	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	NaN	2019.0
1	354020	Ubinas	22324	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	>	2019.0
2	290250	Raikoke	22321	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	NaN	2019.0
3	233020	Fournaise, Piton de la	22318	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	NaN	2019.0
4	311120	Great Sitkin	22322	Confirmed Eruption	NaN	NaN	NaN	NaN	2019	NaN	...	Historical Observations	NaN	2019.0

EXTRACT A SUBSET - USING CONDITIONALS!

e.g extract lat-long coordinates between +/- 40 degrees

Notice the use of square brackets [ ] and the conditional within them
Now we combine two conditional statements

```
data[(data['Column A'] > X ) & (data['Column A'] < Y)]
```

```
# <<-------------------- INSERT CODE HERE ----------------------->>
Latitude = data[(data['Latitude'] > -40) & (data['Latitude'] < 40) ]['Latitude'].values
Longitude = data[(data['Latitude'] > -40) & (data['Latitude'] < 40)]['Longitude'].values
# <<------------------------------------------------------------->>
```

PANDAS AND DATA DISCOVERY

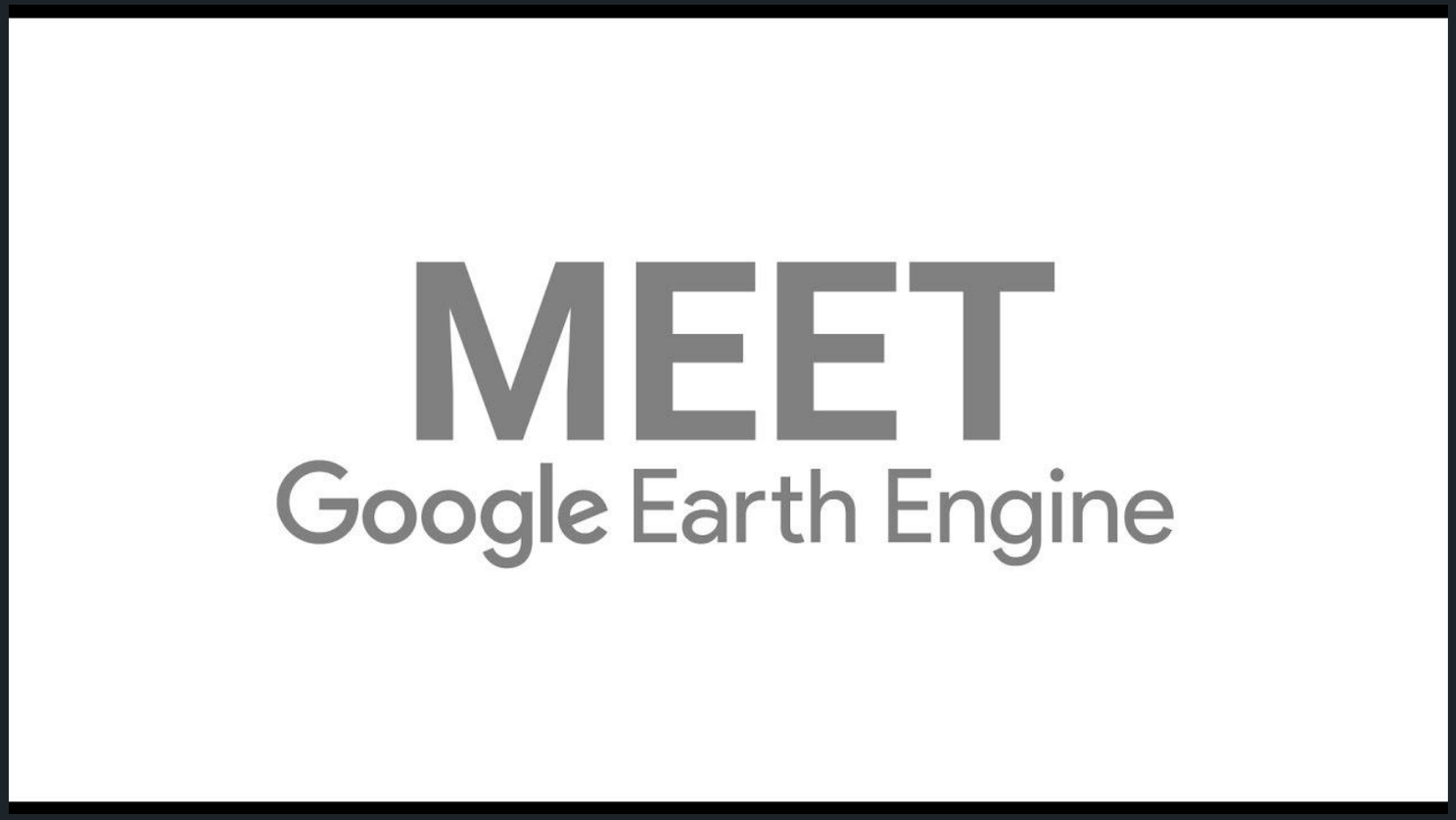


- DATA DATA EVERYWHERE.....IN WALKS PANDAS
- GEOSPATIAL DATA
- PRACTICAL 6



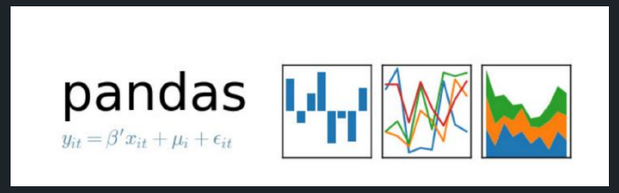
David Topping: david.topping@manchester.ac.uk

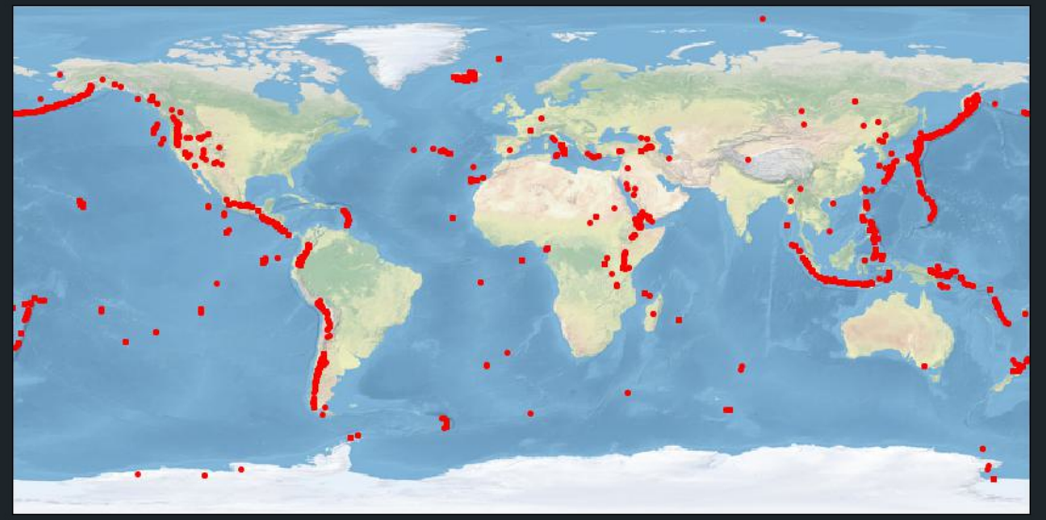
WHAT CAN WE DO WITH GEOSPATIAL DATA?



WHAT CAN WE DO WITH GEOSPATIAL DATA?







LOAD CSV FILE → EXTRACT COORDINATES OF ERUPTIONS → PLOT

```python
import cartopy.crs as ccrs
import matplotlib.pyplot as plt

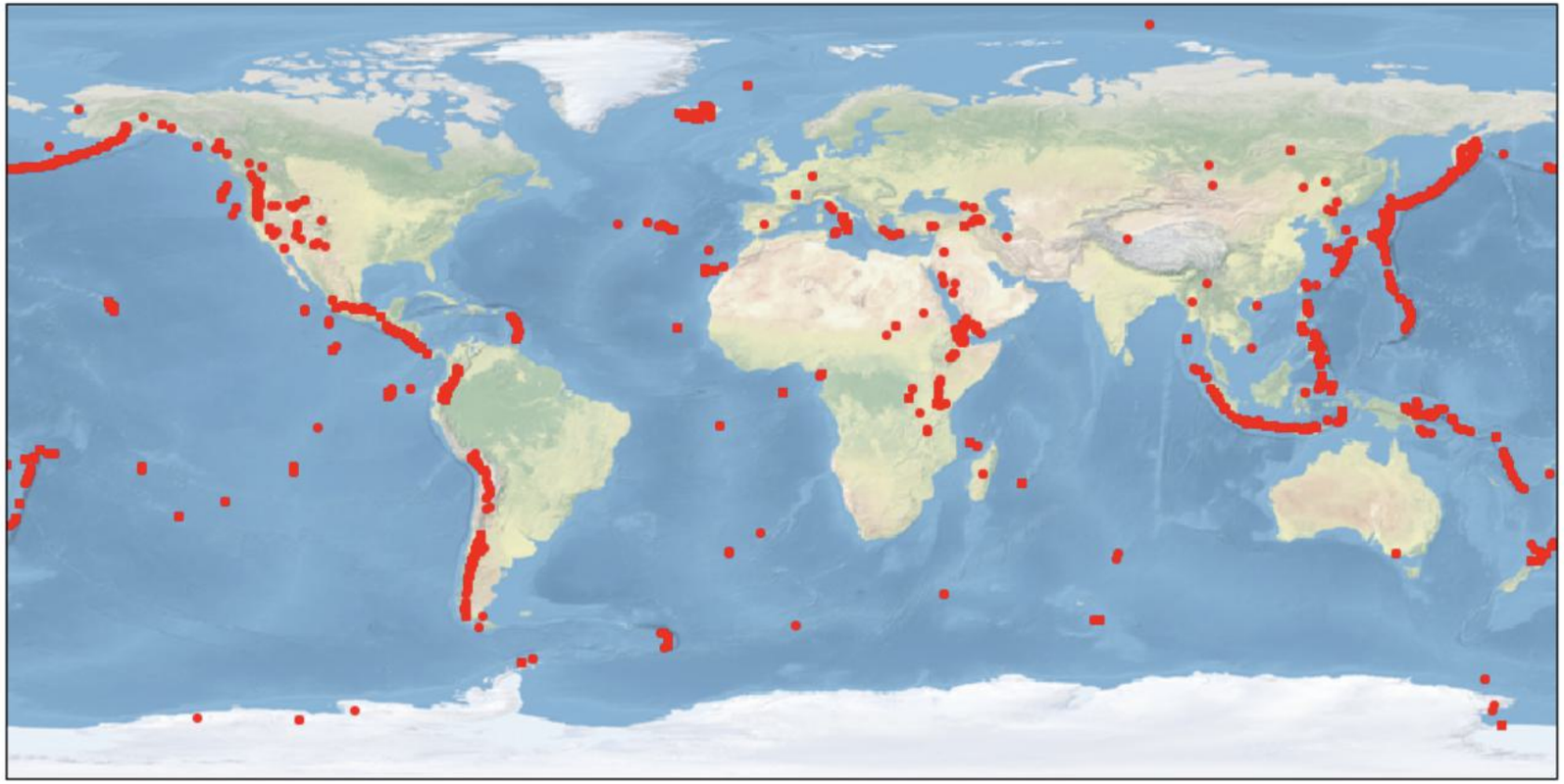
fig = plt.figure(figsize=(15,15))

ax = plt.axes(projection=ccrs.PlateCarree())
ax.stock_img()

Latitude = data['Latitude'].values # Extract the 'Latitude' column and then its .values
Longitude = data['Longitude'].values # We can do the same with Longitude values

plt.scatter(Longitude, Latitude,
          color='red', linewidth=2, s=9, marker='o',
          transform=ccrs.Geodetic(),
          )

plt.show()
```



IT COMBINES WITH MATPLOTLIB AND WE PROVIDE YOU WITH THE TEMPLATE FOR THESE PLOTS. YOU DO NOT NEED TO REMEMBER THESE

character	description
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker

```
# <<-------------------- INSERT CODE HERE ----------------------->>
Latitude = data[(data['Latitude'] > -40) & (data['Latitude'] < 40) ]['Latitude'].values # Extract th
Longitude = data[(data['Latitude'] > -40) & (data['Latitude'] < 40)]['Longitude'].values # We can do
# <<------------------------------------------------------------->>

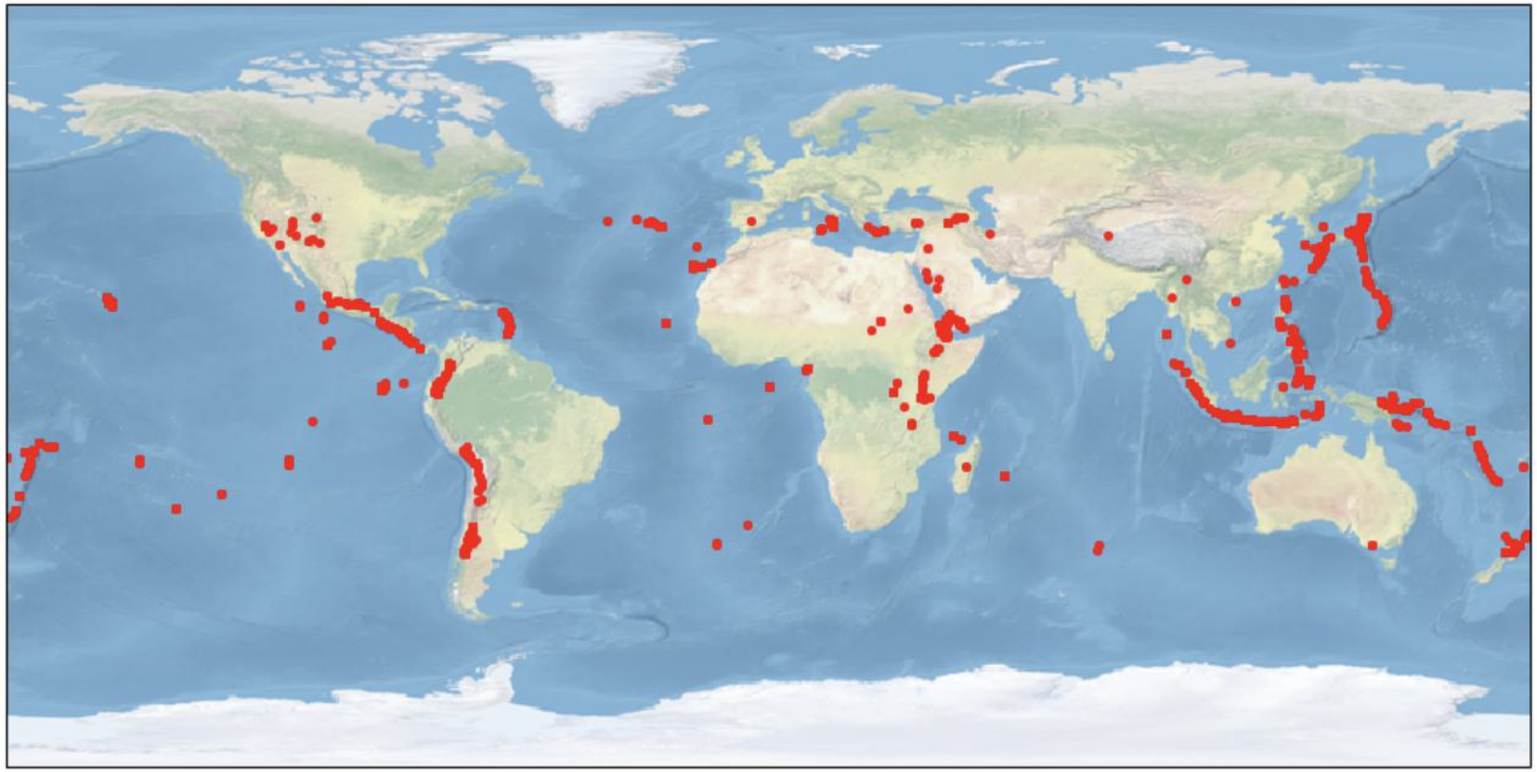
import cartopy.crs as ccrs
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(15,15))

ax = plt.axes(projection=ccrs.PlateCarree())
ax.stock_img()

plt.scatter(Longitude, Latitude,
          color='red', linewidth=2, s=9, marker='o',
          transform=ccrs.Geodetic(),
          )

plt.show()
```



WE CAN PLOT A SUBSET OF DATA!

DOES THE SUBSET SELECTION METHOD MAKE SENSE?

PANDAS AND DATA DISCOVERY

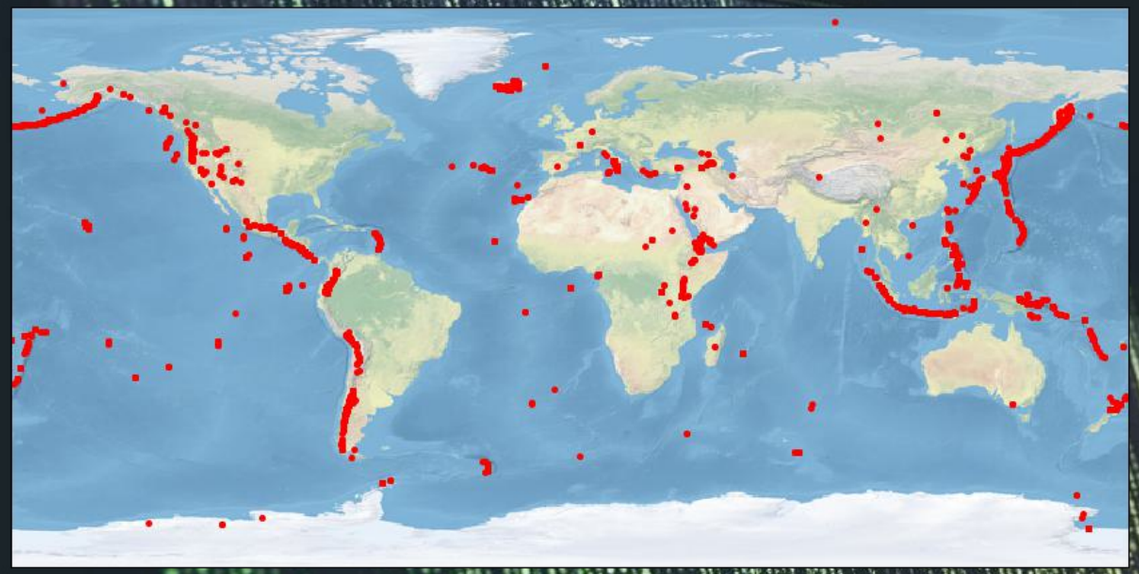


- DATA DATA EVERYWHERE….IN WALKS PANDAS
- GEOSPATIAL DATA
- PRACTICAL 6



David Topping: david.topping@manchester.ac.uk

LECTURE 6



SUMMARY

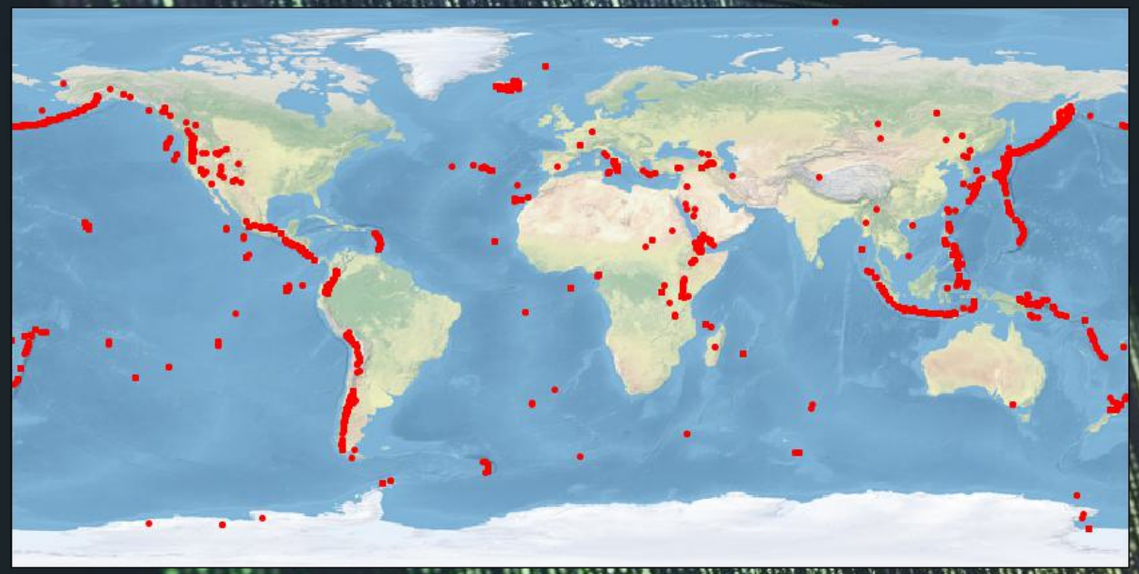
INTERACTING WITH DATA IN PYTHON

- Data is everywhere.
- Pandas is a powerful module
- Pandas has internal functions and plotting capability
- We need to select data according to specific criteria - Pandas can do this for us

GEOSPATIAL PLOTTING

- Cartopy module allows us to plot data with coordinates

PRACTICAL 6



GOALS

CODE

- Become familiar with the Pandas package
- Select a subset of data according to some criteria being met
- Generate histograms

GENERAL EXPERIENCE

- Start to develop skills to interact with data - highly valuable
- Increase the diversity of problems you can solve